DA3 · SECTION 29

Outbox pipeline — Oracle OLTP into TimescaleDB OLAP

Every meter tick is written **twice inside one transaction**: the billing row that decides money, and an outbox row that decides analytics. A relay drains the outbox into the TimescaleDB sink, where the primary key `(session_id, ts)` makes a replay harmless — **at-least-once delivery plus an idempotent sink**.



CRASH SAFETY Killed between the **COPY** and the commit? The marks roll back, the events stay unprocessed, and the sink dedupes them on the primary key when the relay picks them up again. Nothing is lost, nothing is counted twice, and the analytics side never holds a lock the money path needs.

money path · durable store · derived / policy

DA3 · SECTION 26

TimescaleDB schema — hypertables, aggregates, policies

Three raw hypertables, three incremental continuous aggregates, and one reference table projected from Oracle. Retention and compression are **policies on the database**, not cron jobs in an application that can be down.

RAW HYPERTABLES

meter_tick

ts TIMESTAMPTZ · 1-day chunks
session_id, connector_ref
meter_kwh, power_kw, voltage_v, current_a
PK (session_id, ts) = idempotency

connector_state_event

ts TIMESTAMPTZ · 1-day chunks
connector_ref, from_state, to_state
cause, session_id · PK (connector_ref, ts)

session_metric

ts TIMESTAMPTZ · 7-day chunks
session_id, station_id
energy_kwh, duration_s, peak_kw

roll up →

CONTINUOUS AGGREGATES

tick_1m

time_bucket 1 min · delta_kwh, avg_kw, peak_kw
refreshed every minute, incrementally

tick_1h

time_bucket 1 h · energy_kwh, avg_kw, peak_kw
a continuous aggregate over a continuous aggregate

state_1m

dominant_state per minute
fault and offline transition counts

← joins in

REFERENCE

station_map

connector_ref -> station_id
standard_code, max_power_kw
projected from Oracle, read-only here

why a copy

the analytics engine must answer a query without a round trip to the money database

compression
meter_tick older than 7 days · segmentby connector_ref

retention
meter_tick older than 90 days · dropped by chunk

retention
connector_state_event older than 180 days

WHY THIS SHAPE Chunk keys are time, so every dashboard query touches a bounded set of chunks instead of the whole history. Aggregates are **incremental**, so a refresh costs the new buckets only. And because the sink dedupes on the primary key, the relay can replay a batch without a reconciliation job.